SWE 402 - Game Development

Assignment 2: 2D Roguelike Game

Overview

In this assignment, you will complete the Unity Learn "Create a 2D Roguelike Game" course to build a turn-based roguelike with procedural level generation, enemies, and resource management. After completing the course, you will extend the project with audio, visual effects, smooth movement, object pooling, and multi-platform builds.

Course Link: https://learn.unity.com/course/2d-roguelike-tutorial?version=6.3

Section 1: Project Setup & Game Board

Complete: "Set up your project" + "Add a game board"

- Create a new Unity project using Editor 6.x LTS and the Universal 2D template
- Import the provided Roguelike2D tutorial assets package
- Set the Main Camera to Orthographic, size 5, solid black background
- Create a Tile Palette with ground tiles from the provided Sprite Sheets
- Write a **BoardManager** script that procedurally generates an 8×8 tilemap using nested for-loops
- Use **Tile[]** arrays and **Random.Range()** to randomly assign ground and wall tiles
- Add border detection logic so edge cells use wall tiles
- Create a **CellData** nested class with a **Passable** boolean; store board state in a **CellData[,]** 2D array
- Set up Unity Version Control and check in your initial changes

Section 2: Player Character & Movement

Complete: "Add a player character"

- Create a PlayerCharacter GameObject from a provided sprite; set **Order in Layer** for correct rendering
- Write a **PlayerController** script that reads arrow key input via **Input.GetKeyDown()**
- Implement grid-based (cell-to-cell) movement with passability checks using **GetCellData()**
- Create a **Spawn()** method to initialize the player at a given cell coordinate
- Refactor BoardManager: rename **Start()** to **Init()** and make it public for external initialization

Section 3: Turn System & Game Architecture

Complete: "Add a turn system"

- Create a **TurnManager** as a pure C# class (non-MonoBehaviour) with a constructor and **Tick()** method
- Create a **GameManager** MonoBehaviour that initializes TurnManager, BoardManager, and PlayerController
- Implement the **Singleton pattern** on GameManager using **static Instance** with **public get; private set;**
- Use **Awake()** for singleton enforcement and **Start()** for game initialization
- Call **TurnManager.Tick()** from PlayerController each time the player moves

Section 4: Food Resource & Inheritance

Complete: "Add a food resource"

- Create a **CellObject** base class with a **virtual PlayerEntered()** method
- Create a **FoodObject** subclass that **overrides PlayerEntered()** to grant food and destroy itself
- Store a **ContainedObject** reference in CellData so cells know what objects they hold
- Add food tracking to GameManager; decrease food each turn, increase on collection
- Use **UI Toolkit** (UI Builder window) to create a FoodLabel that displays the current food count
- Create multiple food prefabs with different **AmountGranted** values; randomize spawns at level start

Section 5: Obstacles, Win/Lose Conditions & Levels

Complete: "Add obstacles" + "Add win and lose conditions"

- Create a **WallObject** subclass of CellObject that changes the tile sprite and blocks player movement
- Implement **PlayerWantsToEnter()** pattern — walls take damage on bump and are destroyed at 0 HP
- Create an **ExitCellObject** subclass that triggers level completion when the player enters its cell
- Implement level progression: clear the board and regenerate a new level on exit
- Add a **Game Over** state when food reaches 0 — disable input and display a Game Over panel via UI Toolkit
- Implement game restart functionality on key press to return to level 1

Section 6: Animation & Enemies

Complete: "Animation" + "Add an enemy"

- Add an **Animator** component to the player with an **AnimatorController** containing Idle, Walk, and Attack states
- Create sprite-based keyframe animations using the Animation window; set sample rate appropriately
- Configure state transitions with proper **Has Exit Time**, **Transition Duration = 0**, and Bool/Trigger conditions
- Control animations from code using **SetBool()** and **SetTrigger()**
- Create an **Enemy** class inheriting from CellObject that subscribes to **TurnManager.OnTick** using **+=** and unsubscribes in **OnDestroy()** using **-=**
- Implement enemy AI: move toward the player each turn; attack (reduce food) if adjacent
- Enemy blocks player entry, takes damage on bump, and is destroyed at 0 HP (same pattern as WallObject)

Section 7: Additional Requirements (Beyond the Course)

After completing all course tutorials, implement **all** of the following enhancements to your project. These integrate concepts from previous missions (M1–M6) that the Roguelike course does not cover.

Audio (M3 concepts)

- Add background music via an **AudioSource** on the Main Camera (loop enabled, reduced volume)
- Add sound effects using **PlayOneShot()** for: player movement, wall attack, food pickup, enemy attack, enemy death, and game over

Visual Effects (M3 concepts)

- Add **ParticleSystem** effects for at least 3 events: wall destruction, food collection, and enemy death
- Control particles from code using **.Play()** and **.Stop()**; instantiate or pool particle prefabs as needed

Smooth Cell Movement (M3–M5 concepts)

- Replace instant grid snapping with smooth movement using a **Coroutine** that lerps the player between cells
- Block input during the movement coroutine to prevent skipping cells

Object Pooling (M6 concepts)

- Implement object pooling for CellObjects (food, walls, enemies) — pre-instantiate a pool at game start
- On level transition, recycle objects via **SetActive(false/true)** instead of Destroy/Instantiate

Code Standards (M6 concepts)

- Use **[SerializeField]** on all private variables exposed to the Inspector — no unnecessary **public** fields
- Use **[Range()]** on numeric tuning variables (e.g., speeds, spawn counts) and **[Tooltip()]** where helpful

Section 8: Build & Submit

Build your completed project for all three platforms and submit.

- Install Windows, macOS, and WebGL build modules via Unity Hub
- Add your scene(s) to **File** → **Build Profiles** → **Scene List**
- Build for **Windows** (standalone .exe), **macOS** (.app), and **WebGL** (index.html + supporting files)
- Test each build to verify it runs correctly outside the Editor
- (Optional) Upload your WebGL build to **Unity Play** or **itch.io** for browser testing

Submission: Submit a **.zip** file or **GitHub repository link** containing:

- Your complete Unity project folder
- A **Builds/** folder with Windows, macOS, and WebGL builds
- A brief **README.md** listing which bonus features (if any) you implemented

---

Bonus Features (Optional — Up to 10 Extra Points)

Implement any of the following for extra credit. Each feature is worth up to 5 points.

- ■ **Dynamic Board Scaling:** Increase board size and/or number of enemies/walls as the player progresses to higher levels
- ■ **Multiple Enemy Types:** Create 2+ enemy subclasses with different AI behaviors (e.g., one chases, one patrols, one ambushes)
- ■ **Items & Stats System:** Add collectible items (weapons, armor, potions) that modify player stats using ScriptableObjects
- ■ **Save & Load System:** Implement data persistence across sessions using JSON serialization (PlayerPrefs or file I/O)
- ■ **Main Menu & Pause Menu:** Add a title screen with Play/Quit buttons and an in-game pause menu with Resume/Restart/Quit
- ■ **Custom Sprites & Theme:** Replace all provided assets with your own sprites or a cohesive custom theme

---

Resources

- Unity Learn Course: Create a 2D Roguelike Game
- Unity 6 Documentation
- Unity Manual: Tilemaps
- Unity Manual: UI Toolkit
- Unity Learn: Introduction to Object Pooling
- Free Assets: Unity Asset Store, OpenGameArt.org, Kenney.nl